

1 0. Includes

```
#include <stdio.h> // printf, FILE, fopen, fgets, perror
#include <stdlib.h> // malloc, realloc, free, exit, strtol
#include <string.h> // strlen, strncpy, strcmp, memset, strchrn
#include <errno.h> // errno
#include <stddef.h> // size_t

#include <unistd.h> // fork, exec, pipe, read, write, close, dup2
#include <sys/types.h> // pid_t
#include <sys/wait.h> // wait, waitpid, WIFEXITED, WEXITSTATUS
#include <fcntl.h> // open, O_RDONLY, O_CREAT, O_TRUNC
#include <signal.h> // signal, sigaction, sig_atomic_t

#include <pthread.h> // pthread_create, pthread_join, mutex, cond
#include <semaphore.h> // sem_t, sem_wait, sem_post
```

2 1. Pointer / Array / String pointer

```
int x = 10;
int *p = &x;
int **pp = &p;

*p = 20; // x = 20
**pp = 30; // x = 30

printf("%p\n", (void *)p);
printf("%d\n", *p);
```

array length

```
void clear(int *xs, size_t n) {
    for (size_t i = 0; i < n; i++) xs[i] = 0;
}

int xs[10];
clear(xs, 10); // pass length separately
```

argv parse

```
char *end;
errno = 0;

long n = strtol(argv[1], &end, 10);

if (errno != 0 || end == argv[1] || *end != '\0') {
    fprintf(stderr, "bad number\n");
    return 1;
}
```

string basics

```
size_t my_strlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0') n++;
    return n;
}

void safe_copy(char *dst, size_t cap, const char *src) {
    if (cap == 0) return;
    strncpy(dst, src, cap - 1);
    dst[cap - 1] = '\0';
}
```

fgets line

```
char line[128];

if (fgets(line, sizeof line, stdin) == NULL) {
    fprintf(stderr, "no input\n");
    return 1;
}

line[strlen(line)] = '\0';

if (strcmp(line, "quit", 4) == 0) {
    // matched
}
```

3 2. Linked List node

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *new_node(int x) {
    Node *n = malloc(sizeof *n);
    if (n == NULL) {
        perror("malloc");
        exit(1);
    }

    n->data = x;
    n->next = NULL;
    return n;
}
```

traverse / search

```
int length(Node *head) {
    int count = 0;

    for (Node *cur = head; cur != NULL; cur = cur->next) {
        count++;
    }
}
```

```
return count;
}

Node *find(Node *head, int x) {
    for (Node *cur = head; cur != NULL; cur = cur->next) {
        if (cur->data == x) return cur;
    }

    return NULL;
}
```

insert front / back

```
void push_front(Node **head, int x) {
    Node *n = new_node(x);
    n->next = *head;
    *head = n;
}

void push_back(Node **head, int x) {
    Node *n = new_node(x);

    if (*head == NULL) {
        *head = n;
        return;
    }

    Node *cur = *head;
    while (cur->next != NULL) cur = cur->next;
    cur->next = n;
}
```

delete normal

```
void delete_value(Node **head, int x) {
    Node *cur = *head;
    Node *prev = NULL;

    while (cur != NULL && cur->data != x) {
        prev = cur;
        cur = cur->next;
    }

    if (cur == NULL) return;

    if (prev == NULL) *head = cur->next;
    else prev->next = cur->next;

    free(cur);
}
```

delete pointer-to-pointer

```
void delete_pp(Node **head, int x) {
    Node **cur = head;

    while (*cur != NULL && (*cur)->data != x) {
        cur = &(*cur)->next;
    }

    if (*cur == NULL) return;

    Node *dead = *cur;
    *cur = dead->next;
    free(dead);
}
```

reverse / free

```
Node *reverse(Node *head) {
    Node *prev = NULL;
    Node *cur = head;

    while (cur != NULL) {
        Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }

    return prev;
}

void free_list(Node *head) {
    while (head != NULL) {
        Node *tmp = head;
        head = head->next;
        free(tmp);
    }
}
```

4 3. Memory / Struct / File

malloc / realloc

```
T *p = malloc(sizeof *p);
if (p == NULL) exit(1);

T *arr = malloc(n * sizeof *arr);
if (arr == NULL) exit(1);

T *tmp = realloc(arr, new_n * sizeof *arr);
if (tmp == NULL) {
    free(arr);
    exit(1);
}
arr = tmp;
```

enum + struct

```
typedef enum {
```

```
STUDENT,
STAFF
} Kind;

typedef struct Person {
    Kind kind;
    int id;
    char name[32];
} Person;
```

text file parse

```
FILE *fp = fopen("input.txt", "r");
if (fp == NULL) {
    perror("fopen");
    exit(1);
}

char line[256];

while (fgets(line, sizeof line, fp) != NULL) {
    int id;
    char name[50];
    float mark;

    if (sscanf(line, "%d %49s %f", &id, name, &mark) == 3) {
        // use values
    }
}

fclose(fp);
```

5 4. Fork / exec / wait
basic fork

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    exit(1);
}

if (pid == 0) {
    // child code
    exit(0);
}

// parent
wait(NULL);
```

waitpid status

```
pid_t pid = fork();

if (pid == 0) {
    exit(7);
}

int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
    printf("exit=%d\n", code);
}
```

create N children

```
pid_t pids[N];

for (int i = 0; i < N; i++) {
    pids[i] = fork();

    if (pids[i] == 0) {
        // child i work
        exit(0);
    }
}

for (int i = 0; i < N; i++) {
    waitpid(pids[i], NULL, 0);
}
```

process chain

```
int my_id = 0;

for (int i = 1; i < n; i++) {
    pid_t pid = fork();

    if (pid == 0) {
        my_id = i;        // child continues
    } else {
        break;           // parent stops
    }
}
```

fork count / exec

```
fork();
fork();
printf("x\n");    // prints 4 times

char *args[] = {"ls", "-l", NULL};
execvp(args[0], args);
perror("execvp");
exit(1);
```

mini shell

```
pid_t pid = fork();

if (pid == 0) {
    char *args[] = {"ls", "-l", NULL};
    execvp(args[0], args);
    perror("execvp");
    exit(1);
}

waitpid(pid, NULL, 0);
```

6 5. Pipe / dup2 / IPC
pipe rules

```
// fd[0] read end, fd[1] write end
// pipe is one-way
```

```
// create pipe before fork
// close unused ends
// EOF only when all write ends closed
```

parent -> child pipe

```
int fd[2];

if (pipe(fd) < 0) {
    perror("pipe");
    exit(1);
}

pid_t pid = fork();

if (pid == 0) {
    close(fd[1]);

    char buf[100];
    ssize_t n = read(fd[0], buf, sizeof buf);

    close(fd[0]);
    exit(0);
}

close(fd[0]);

char msg[] = "hello";
write(fd[1], msg, sizeof msg);

close(fd[1]);
wait(NULL);
```

two-pipe request / response

```
int p2c[2], c2p[2];

pipe(p2c);
pipe(c2p);

pid_t pid = fork();

if (pid == 0) {
    close(p2c[1]);
    close(c2p[0]);

    // read p2c[0], write c2p[1]

    close(p2c[0]);
    close(c2p[1]);
    exit(0);
}

close(p2c[0]);
close(c2p[1]);

// write p2c[1], read c2p[0]

close(p2c[1]);
close(c2p[0]);
wait(NULL);
```

redirect stdout

```
int fd = open("out.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

if (fd < 0) exit(1);

dup2(fd, STDOUT_FILENO);
close(fd);

execlp("ls", "ls", "-l", NULL);
perror("execlp");
exit(1);
```

pipeline: ls — wc -l

```
int fd[2];
pipe(fd);

if (fork() == 0) {
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]);
    close(fd[1]);
    execlp("ls", "ls", NULL);
    exit(1);
}

if (fork() == 0) {
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    close(fd[1]);
    execlp("wc", "wc", "-l", NULL);
    exit(1);
}
```

```
close(fd[0]);
close(fd[1]);

wait(NULL);
wait(NULL);
```

7 6. Thread Sync
thread args

```
typedef struct Args {
    int id;
    int *arr;
    size_t start;
    size_t end;
    long result;
} Args;

void *worker(void *arg) {
    Args *a = arg;
    long sum = 0;

    for (size_t i = a->start; i < a->end; i++) {
        sum += a->arr[i];
    }

    a->result = sum;
    return NULL;
}
```

create / join

```
pthread_t tids[N];
Args args[N];

for (int i = 0; i < N; i++) {
    args[i].id = i;
    pthread_create(&tids[i], NULL, worker, &args[i]);
}

long total = 0;

for (int i = 0; i < N; i++) {
    pthread_join(tids[i], NULL);
    total += args[i].result;
}
```

mutex / semaphore

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&lock);
shared++;
pthread_mutex_unlock(&lock);

sem_t sem;
sem_init(&sem, 0, 1);

sem_wait(&sem);
// critical section
sem_post(&sem);

sem_destroy(&sem);
```

condition variable

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int ready = 0;

pthread_mutex_lock(&lock);

while (!ready) {
    pthread_cond_wait(&cond, &lock);
}

// use resource

pthread_mutex_unlock(&lock);
```

8 7. Checklist

```
// malloc -> free
// fopen -> fclose
// open -> close
// pipe -> close unused ends
// fork -> wait/waitpid
// exec failure -> perror + exit
// mutex -> unlock
// cond -> while(predicate false)
// pthread arg -> separate struct per thread
// array -> pass length separately
// linked list delete -> relink before free
```